TP-DNN

APM_5DS24_TP – Deep Learning II
Institut Polytechnique de Paris

Sergio Magalhães Contente Chenxi Yang

22 Mars 2026

Résumé

Ce rapport présente une implémentation from scratch en NumPy de Machines de Boltzmann Restreintes (RBM) [9], de Réseaux de Croyance Profonds (DBN) [2] et de Réseaux de Neurones Profonds (DNN) pour la classification de chiffres manuscrits sur MNIST [4]. Nous implémentons un pré-entraînement non supervisé par entraînement glouton couche par couche de RBM [8], suivi d'un ajustement supervisé par rétropropagation avec une couche de classification softmax. Une comparaison systématique entre les réseaux pré-entraînés et initialisés aléatoirement est réalisée selon trois axes : la profondeur du réseau, la largeur des couches et la taille de l'ensemble d'entraînement [3]. En bonus, nous comparons cinq modèles génératifs (RBM, DBN, VAE [5], GAN [6], DDPM [7]) sous un budget de paramètres similaire sur MNIST.

Table des matières

1 Introduction **3**
1.1 Exécution du code . . . 3

2 Méthodologie et Architectures **3**
2.1 Machine de Boltzmann Restreinte (RBM) . . . 3
2.1.1 Modèle d'énergie et inférence . . . 3
2.1.2 Apprentissage par Divergence Contrastive (CD-1) . . . 4
2.1.3 Génération par échantillonnage de Gibbs . . . 4
2.2 Réseau de Croyances Profondes (DBN) et Pré-entraînement . . . 4
2.2.1 Architecture par empilement . . . 4
2.2.2 Apprentissage glouton couche par couche . . . 5
2.2.3 Génération par passe descendante . . . 5

3 Validation sur Binary AlphaDigits **5**
3.1 Expériences RBM . . . 5
3.1.1 Données originales . . . 5
3.1.2 Effet du nombre d'unités cachées . . . 6
3.1.3 Effet de la diversité des caractères . . . 7
3.2 Expériences DBN . . . 7
3.2.1 Effet de la profondeur du réseau . . . 7
3.2.2 Effet de la diversité des caractères sur le DBN . . . 8

4 Étude sur MNIST **8**
4.1 Protocole expérimental . . . 8
4.2 Expérience 1 : Taux d'erreur en fonction du nombre de couches cachées . . . 8
4.3 Expérience 2 : Taux d'erreur en fonction du nombre de neurones par couche . . . 9
4.4 Expérience 3 : Taux d'erreur en fonction du nombre d'échantillons d'entraînement . . . 10
4.5 Configuration du meilleur modèle . . . 11

5 Bonus : Comparaison de modèles génératifs **12**

6 Conclusion **13**

1 Introduction

L'objectif de ce projet est d'implémenter un réseau de neurones profond pré-entraîné pour la classification de chiffres manuscrits et d'évaluer l'apport du pré-entraînement non supervisé par rapport à une initialisation aléatoire. L'implémentation est entièrement réalisée en NumPy, suivant l'architecture où un RBM [9] est la brique de base d'un DBN [2], qui fournit à son tour les poids pré-entraînés pour un DNN avec une couche de classification softmax supplémentaire.

Nous procédons en trois étapes :

1. Validation du RBM et du DBN sur le jeu de données Binary AlphaDigits (Section 3) ;
2. Construction du DNN avec pré-entraînement et rétropropagation, puis étude systématique sur MNIST (Section 4) ;
3. Comparaison de modèles génératifs en bonus (Section 5).

1.1 Exécution du code

Le code est organisé dans le package `src/` (modules `rbm.py`, `dbn.py`, `dnn.py`, `utils.py`).

Mise en place des données. Les jeux de données ne sont pas inclus dans le dépôt et doivent être placés manuellement dans le répertoire `data/` :

```
data/
  binaryalphadigs.mat                # Binary AlphaDigits (.mat)
  Fichiers MNIST-20260320/           # Archive MNIST dezippee
      train-images-idx3-ubyte
      train-labels-idx1-ubyte
      t10k-images-idx3-ubyte
      t10k-labels-idx1-ubyte
```

— Placer le fichier `binaryalphadigs.mat` directement dans `data/`.
— Dézipper l'archive MNIST fournie avec le sujet dans `data/FichiersMNIST-20260320/` (les 4 fichiers IDX bruts doivent se trouver à la racine de ce sous-répertoire).

Exécution. Pour reproduire les expériences :

```
pip install numpy matplotlib scipy
python experiments/principal_RBM_alpha.py   # RBM sur AlphaDigits
python experiments/principal_DBN_alpha.py   # DBN sur AlphaDigits
python experiments/principal_DNN_MNIST.py   # Etude MNIST (Figures 1-3)
python experiments/principal_best_model.py  # Meilleur modele
```

Le notebook bonus (`notebooks/bonus_comparison.ipynb`) est prévu pour Google Colab avec GPU.

2 Méthodologie et Architectures

2.1 Machine de Boltzmann Restreinte (RBM)

2.1.1 Modèle d'énergie et inférence

Une RBM [9] est un modèle génératif paramétré par $\theta = \{W, a, b\}$, où $W \in \mathbb{R}^{p\times q}$ est la matrice de poids, $a \in \mathbb{R}^p$ le biais des unités visibles v, et $b \in \mathbb{R}^q$ le biais des unités cachées h. L'indépendance conditionnelle des couches permet de calculer les probabilités d'activation bidirectionnelles via la fonction sigmoïde $\sigma(\cdot)$:

$$P(h_j = 1|v) = \sigma\left(b_j + \sum_{i=1}^{p} v_i W_{ij}\right) \tag{1}$$

$$P(v_i = 1|h) = \sigma\left(a_i + \sum_{j=1}^{q} h_j W_{ij}\right) \tag{2}$$

Correspondance dans l'implémentation :

- `init_RBM(p,q)` : Initialise les paramètres. W suit une loi $\mathcal{N}(0, 0.01^2)$, a et b sont initialisés à 0.
- `sigmoid(x)` : Implémente $\sigma(\cdot)$ avec un `np.clip(x,-500,500)` pour éviter les instabilités numériques (overflow).
- `entree_sortie_RBM` : Implémente l'Équation 1 sous forme vectorisée (`V@rbm["W"]+rbm["b"]`).
- `sortie_entree_RBM` : Implémente l'Équation 2 sous forme vectorisée (`H@rbm["W"].T+rbm["a"]`).

2.1.2 Apprentissage par Divergence Contrastive (CD-1)

L'optimisation non supervisée vise à maximiser la log-vraisemblance [1]. Le gradient analytique étant intraitable, il est approximé par l'algorithme CD-1. Pour un mini-batch d'entrée $v^{(0)}$ de taille bs, les règles de mise à jour avec le taux d'apprentissage η sont :

$$\Delta W = \frac{\eta}{bs}\left(v^{(0)T} P(h^{(0)}|v^{(0)}) - v^{(1)T} P(h^{(1)}|v^{(1)})\right) \tag{3}$$

$$\Delta a = \frac{\eta}{bs}\sum\left(v^{(0)} - v^{(1)}\right), \quad \Delta b = \frac{\eta}{bs}\sum\left(P(h^{(0)}|v^{(0)}) - P(h^{(1)}|v^{(1)})\right) \tag{4}$$

Correspondance dans l'implémentation (`train_RBM`) :

- **Phase positive :** Calcul des probabilités `h0_prob` et échantillonnage de `h0_sample` via une distribution de Bernoulli.
- **Phase négative :** Reconstruction de `v1_prob` à partir de `h0_sample`, puis recalcul des probabilités cachées `h1_prob`.
- **Mise à jour :** L'incrémentation de la matrice se fait vectoriellement : `rbm["W"]+=lr*(v0.T@h0_prob-v1_prob.T@h1_prob)/bs`.
- **Métrique :** L'erreur de reconstruction (MSE) est calculée par `np.mean((v0-v1_prob)**2)` à la fin de chaque batch.

2.1.3 Génération par échantillonnage de Gibbs

La RBM agit comme un générateur stochastique en simulant une Chaîne de Markov Monte Carlo (MCMC). À partir d'un bruit initial $v^{(0)}$, l'échantillonneur de Gibbs alterne les inférences pendant k itérations :

$$h^{(t)} \sim P(h|v^{(t-1)}) \quad \text{suivi de} \quad v^{(t)} \sim P(v|h^{(t)}) \tag{5}$$

La distribution de $v^{(k)}$ converge asymptotiquement vers la distribution apprise par le modèle.

Correspondance dans l'implémentation (`generer_image_RBM`) :

- **Initialisation :** Les unités visibles sont initialisées avec un bruit uniforme binaire : `v=(np.random.rand(n_images,p)>0.5)`.
- **Boucle de Markov :** La boucle `for_inrange(n_gibbs)`: alterne séquentiellement les appels à `entree_sortie_RBM` et `sortie_entree_RBM`, en binarisant systématiquement les états h et v par échantillonnage aléatoire (`<prob`).

2.2 Réseau de Croyances Profondes (DBN) et Pré-entraînement

2.2.1 Architecture par empilement

Un Réseau de Croyances Profondes (DBN) [2] est un modèle génératif multicouche construit par l'empilement séquentiel de Machines de Boltzmann Restreintes. Structurellement, la couche cachée de la RBM de niveau ℓ agit comme la couche visible de la RBM de niveau $\ell + 1$.

Correspondance dans l'implémentation (`init_DBN`) :

— L'architecture globale est définie par un vecteur de dimensions (`layer_sizes`).
— Le DBN est matérialisé par une liste contenant $L-1$ structures RBM indépendantes, instanciées séquentiellement par `init_RBM(layer_sizes[i],layer_sizes[i+1])`.

2.2.2 Apprentissage glouton couche par couche

L'optimisation globale d'un modèle profond étant particulièrement instable, le DBN bénéficie d'une procédure de pré-entraînement glouton ascendant [8]. Chaque RBM est entraînée localement et indépendamment via l'algorithme CD-1. Une fois la RBM de niveau ℓ convergée, les probabilités d'activation de ses unités cachées sont extraites et servent de données d'entrée $v^{(\ell+1)}$ pour l'entraînement de la RBM de niveau $\ell + 1$:

$$v^{(\ell+1)} = P(h^{(\ell)} = 1|v^{(\ell)}; W^{(\ell)}, b^{(\ell)}) \tag{6}$$

Correspondance dans l'implémentation (`train_DBN`) :
— **Boucle d'entraînement :** Le processus itère sur chaque RBM de la liste (`fori,rbminenumerate(dbn)`).
— **Optimisation locale :** Fait appel à `train_RBM` pour l'optimisation non supervisée des poids locaux.
— **Propagation ascendante :** Met à jour l'espace de représentation via `current_data=entree_sortie_RBM(rbm,current_data)` pour constituer le jeu de données de la couche suivante.

2.2.3 Génération par passe descendante

Le processus génératif d'un DBN s'opère en deux phases : une exploration stochastique dans l'espace latent le plus profond, suivie d'une projection déterministe vers l'espace visible.
— **Phase 1 (Équilibre MCMC) :** Un échantillonnage de Gibbs alterné est exécuté exclusivement sur la RBM supérieure pour échantillonner à partir de la distribution stationnaire latente.
— **Phase 2 (Passe ancestrale) :** L'information est propagée de manière descendante à travers les poids transposés $W^{(\ell)T}$ des couches inférieures pour reconstruire l'image finale.
Correspondance dans l'implémentation (`generer_image_DBN`) :
— **Niveau supérieur :** Échantillonnage de Gibbs sur le dernier bloc `top_rbm=dbn[-1]` pendant `n_gibbs` itérations avec binarisation stricte.
— **Passe descendante :** Une boucle inversée (`range(len(dbn)-2,-1,-1)`) propage l'information vers le bas. L'appel exclusif à `sortie_entree_RBM` (utilisation des probabilités continues plutôt que des échantillons binaires) agit comme une régularisation, générant des images plus lisses et moins bruitées.

3 Validation sur Binary AlphaDigits

Avant d'aborder MNIST, nous validons les implémentations du RBM et du DBN sur le jeu de données Binary AlphaDigits, qui contient de petites images binaires (20×16) de chiffres et de lettres (36 classes, 39 échantillons chacune).

3.1 Expériences RBM

3.1.1 Données originales

Nous entraînons sur les caractères 0, 1 et 2 (117 échantillons au total). La Figure 1 montre des échantillons représentatifs.

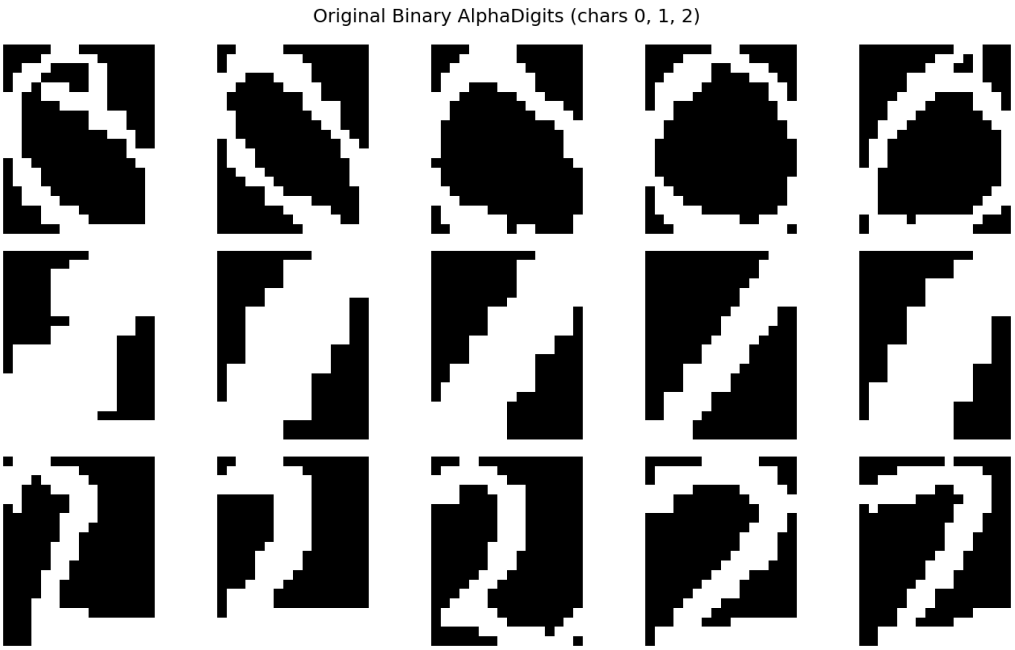
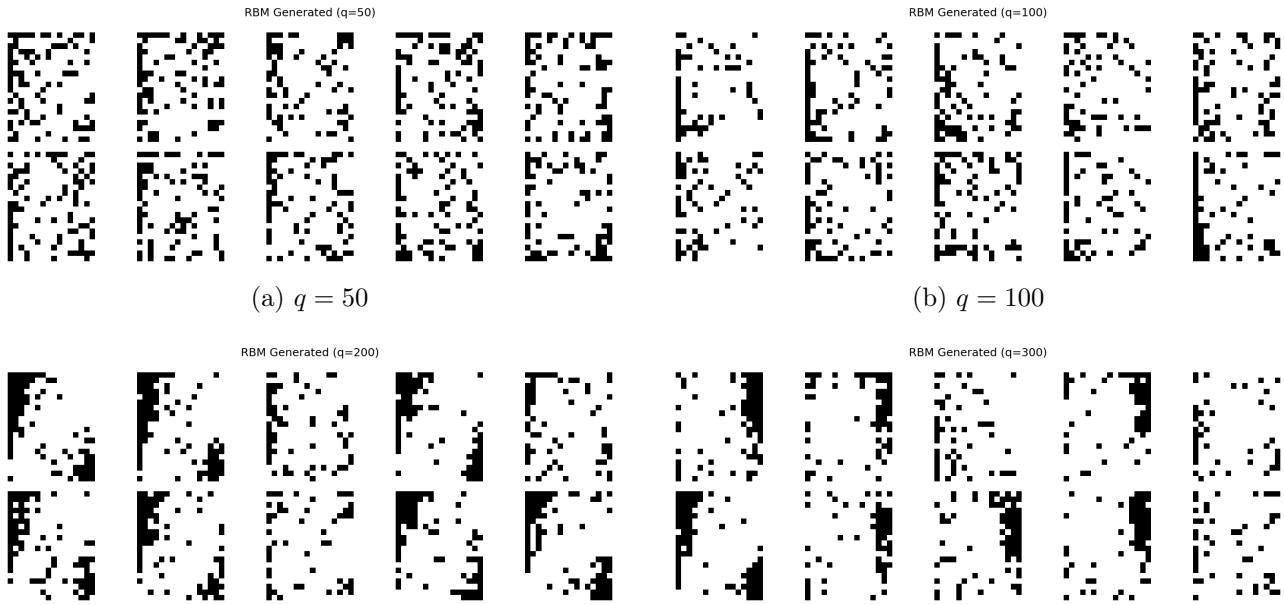


FIGURE 1 – Échantillons originaux de Binary AlphaDigits (caractères 0, 1, 2).

3.1.2 Effet du nombre d'unités cachées

Nous entraînons des RBM avec $q \in \{50, 100, 200, 300\}$ unités cachées (100 époques, $\eta = 0.1$, taille de batch 32) en utilisant l'algorithme de Divergence Contrastive (CD-1) [1] et générons des images via 500 pas de Gibbs.



(a) $q = 50$ (b) $q = 100$

(c) $q = 200$ (d) $q = 300$

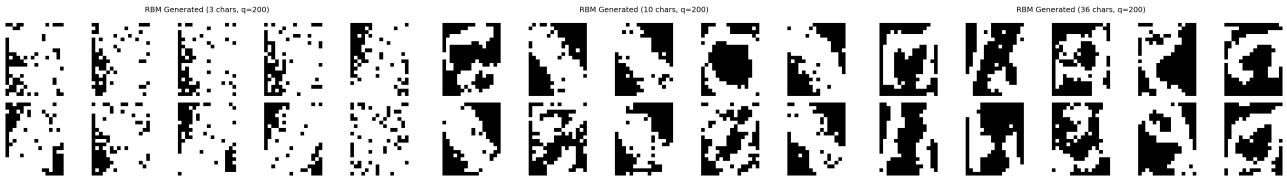
FIGURE 2 – Images générées par le RBM en fonction du nombre d'unités cachées q.

Avec $q = 50$, les images générées sont très bruitées avec quasiment aucune structure discernable, indiquant une capacité insuffisante pour capturer la distribution des données. À $q = 100$, une certaine structure locale commence à émerger : des régions noires contiguës et des contours deviennent visibles, bien que les images restent bruitées. Avec $q = 200$ et $q = 300$, les images générées montrent des structures de blocs plus nettes rappelant les caractères d'entraînement, avec des traits verticaux et diagonaux reconnaissables. L'amélioration de 200 à 300 est marginale, suggérant que $q = 200$ offre un

bon compromis entre capacité et la taille limitée de l'ensemble d'entraînement (117 échantillons). Néanmoins, la nature stochastique binaire du processus d'échantillonnage de Gibbs produit intrinsèquement des sorties bruitées.

3.1.3 Effet de la diversité des caractères

Nous fixons $q = 200$ et varions le nombre de caractères utilisés pour l'entraînement : 3, 10 et 36 (tous les caractères alphanumériques).



(a) 3 caractères (b) 10 caractères (c) 36 caractères

FIGURE 3 – Images générées par le RBM en fonction du nombre de caractères d'entraînement ($q = 200$).

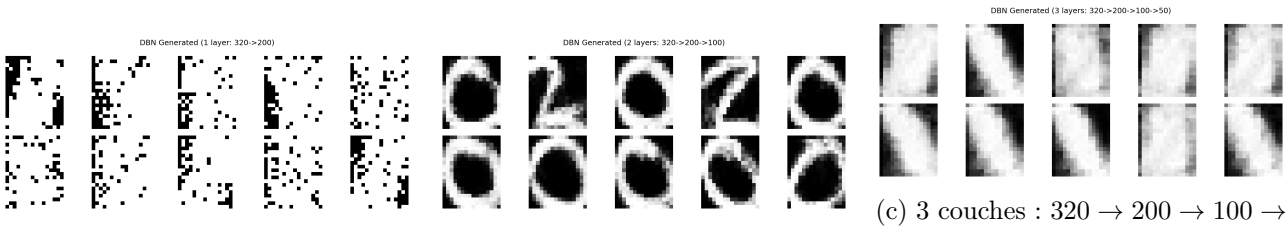
Avec 3 caractères, les images générées sont très bruitées avec des pixels épars et peu de structure cohérente, en raison du très petit ensemble d'entraînement (117 échantillons). Avec 10 caractères (390 échantillons), la qualité s'améliore nettement : les images présentent des structures de blocs claires et des formes de caractères reconnaissables, le volume de données supplémentaire permettant au RBM d'apprendre des caractéristiques plus robustes.

Avec 36 caractères (1 404 échantillons), deux effets antagonistes coexistent. D'une part, le bruit pixelélisé disparait grâce au volume de données accru. D'autre part, la capacité du modèle ($q = 200$ unités cachées) devient insuffisante pour séparer correctement les 36 modes de la distribution. Les images générées apparaissent alors comme des **mélanges hybrides** de plusieurs caractères — des "caractères fantômes" qui combinent des traits de différentes classes plutôt que de reproduire un caractère distinct. Ce phénomène de *mélange de modes* illustre la limitation fondamentale du RBM : lorsque le nombre de classes dépasse la capacité représentationnelle du modèle, celui-ci moyenne entre les modes au lieu de les séparer. Le meilleur compromis se situe autour de 10 caractères, où le RBM dispose à la fois de suffisamment de données pour apprendre des caractéristiques robustes et de suffisamment de capacité pour distinguer les classes.

3.2 Expériences DBN

3.2.1 Effet de la profondeur du réseau

Nous entraînons des DBN [2] avec 1, 2 et 3 couches cachées sur les caractères 0, 1, 2, en utilisant l'entraînement glouton couche par couche [8].



(a) 1 couche : 320 → 200 (b) 2 couches : 320 → 200 → 100 (c) 3 couches : 320 → 200 → 100 → 50

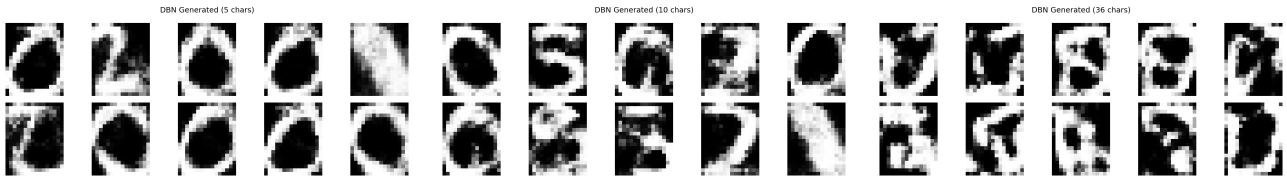
FIGURE 4 – Images générées par le DBN avec une profondeur variable.

Le DBN à une seule couche (équivalent à un RBM) produit des images binaires bruitées, similaires à celles du RBM isolé. Le **DBN à 2 couches** produit les meilleurs résultats : la propagation descendante à travers deux couches de probabilités apprises génère des images en niveaux de gris lisses avec des formes de chiffres clairement reconnaissables (les 0 et les 2 sont distinctement visibles). Cela démontre

l'avantage de l'apprentissage hiérarchique de caractéristiques — la première couche capture la structure de bas niveau tandis que la seconde capture des abstractions de plus haut niveau. Le DBN à 3 couches, en revanche, produit des images en niveaux de gris floues et moins distinctes. Le RBM supérieur opère sur seulement 50 unités cachées, ce qui limite l'expressivité de l'échantillonnage de Gibbs et conduit à des représentations de la couche supérieure moins informatives propagées vers le bas.

3.2.2 Effet de la diversité des caractères sur le DBN

Nous fixons l'architecture à $320 \to 200 \to 100$ et varions le nombre de caractères.



(a) 5 caractères (b) 10 caractères (c) 36 caractères

FIGURE 5 – Images générées par le DBN en fonction du nombre de caractères d'entraînement.

Le DBN à 2 couches produit des images en niveaux de gris lisses pour tous les niveaux de diversité. Avec 5 caractères, les images générées sont les plus reconnaissables. Lorsque le nombre de caractères augmente à 36, les images générées deviennent plus uniformes et moins spécifiques à une classe, car le modèle moyenne sur une distribution plus large. Comparé au RBM isolé, la structure hiérarchique du DBN et la propagation descendante basée sur les probabilités produisent systématiquement des sorties plus lisses et plus reconnaissables, confirmant l'avantage des modèles génératifs profonds même sur ce petit jeu de données.

4 Étude sur MNIST

Nous passons maintenant à l'étude principale : comparer un DNN pré-entraîné (pré-entraînement non supervisé par RBM + rétropropagation) à un DNN initialisé aléatoirement (rétropropagation uniquement) sur MNIST.

4.1 Protocole expérimental

Les images MNIST sont binarasées au seuil 127. Pour chaque expérience, deux DNN identiques sont créés. L'un est pré-entraîné par entraînement glouton couche par couche de RBM (100 époques par couche), puis ajusté par rétropropagation (200 époques). L'autre est entraîné à partir d'une initialisation aléatoire avec rétropropagation uniquement (200 époques). Les deux utilisent un taux d'apprentissage $\eta = 0.1$ et une taille de batch de 128. Le taux d'erreur de classification sur l'ensemble de test est rapporté.

Trois expériences sont réalisées conformément au sujet. Nous analysons successivement l'évolution de la précision du modèle en fonction de la complexité architecturale à travers la profondeur (nombre de couches) et la largeur (nombre de neurones par couche) du réseau, ainsi que sa robustesse face à la variation du volume de données d'apprentissage disponibles. Chaque test est réalisé en maintenant les autres paramètres constants afin de garantir la pertinence de la comparaison et d'identifier les régimes où le pré-entraînement apporte le gain de performance le plus significatif.

4.2 Expérience 1 : Taux d'erreur en fonction du nombre de couches cachées

Nous fixons le nombre de neurones par couche cachée à 200 et varions le nombre de couches cachées de 2 à 5. L'architecture est $[784, 200, \ldots, 200, 10]$.

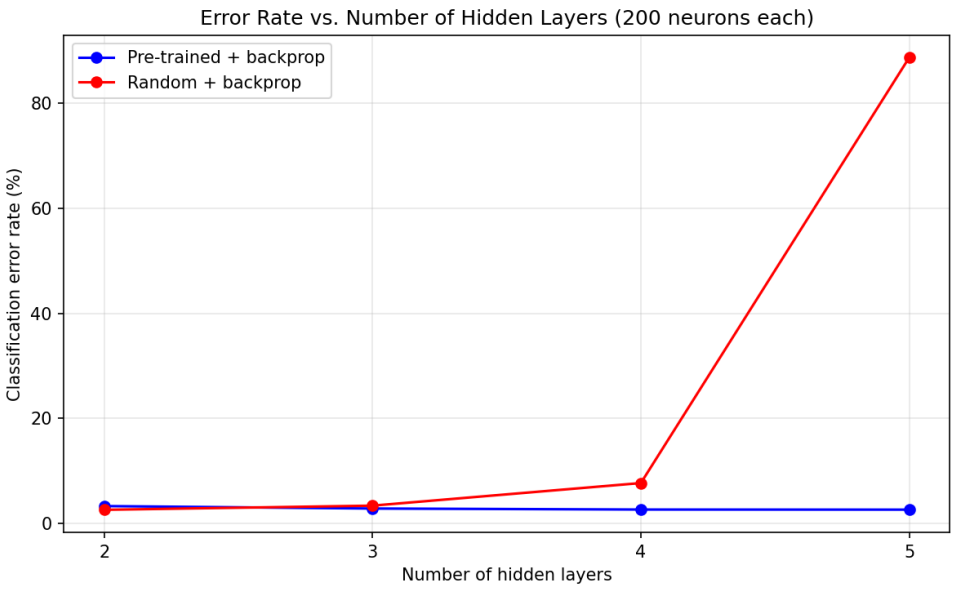


FIGURE 6 – Taux d'erreur de classification sur l'ensemble de test en fonction du nombre de couches cachées (200 neurones chacune). Bleu : pré-entraîné + rétropropagation. Rouge : initialisation aléatoire + rétropropagation.

TABLE 1 – Taux d'erreur en fonction du nombre de couches cachées (200 neurones chacune).

Couches cachées	Erreur test (%)		Erreur entraîn. (%)	
	Pré-entraîné	Aléatoire	Pré-entraîné	Aléatoire
2	3.31	**2.60**	0.22	0.01
3	**2.84**	3.39	0.12	0.07
4	**2.65**	7.69	0.07	5.37
5	**2.63**	88.65	0.06	88.76

Analyse. Avec 2 couches cachées, le réseau aléatoire surpasse légèrement le réseau pré-entraîné (2.60 % contre 3.31 %). Cependant, lorsque la profondeur augmente, le réseau initialisé aléatoirement se dégrade de manière dramatique : à 4 couches il atteint 7.69 % d'erreur, et à 5 couches la perte reste bloquée à ≈ 2.30 ($= \ln 10$, c'est-à-dire une prédiction aléatoire) pendant les 200 époques d'entraînement, donnant ≈ 90 % d'erreur.

Ceci est une démonstration classique du **problème du gradient évanescent** [3] : dans les réseaux profonds initialisés aléatoirement, les gradients deviennent extrêmement petits dans les couches antérieures, empêchant tout apprentissage. Le plateau de la perte à $\ln(10)$ confirme que le réseau produit des probabilités softmax uniformes. En revanche, le réseau pré-entraîné maintient des performances stables et même légèrement améliorées lorsque la profondeur augmente (2.65 % à 4 couches), car le pré-entraînement par RBM place les poids dans une région favorable de l'espace des paramètres où les gradients circulent efficacement.

Le point clé est que le **pré-entraînement est essentiel pour les architectures profondes** (plus de 2–3 couches cachées). Pour les réseaux peu profonds (2 couches), l'initialisation aléatoire est compétitive. Pour notre meilleur modèle, nous sélectionnons **2 couches cachées** car les réseaux plus profonds n'améliorent pas l'erreur de test malgré le pré-entraînement.

4.3 Expérience 2 : Taux d'erreur en fonction du nombre de neurones par couche

Nous fixons l'architecture à 2 couches cachées et varions le nombre de neurones par couche : 100, 300, 500 et 700.

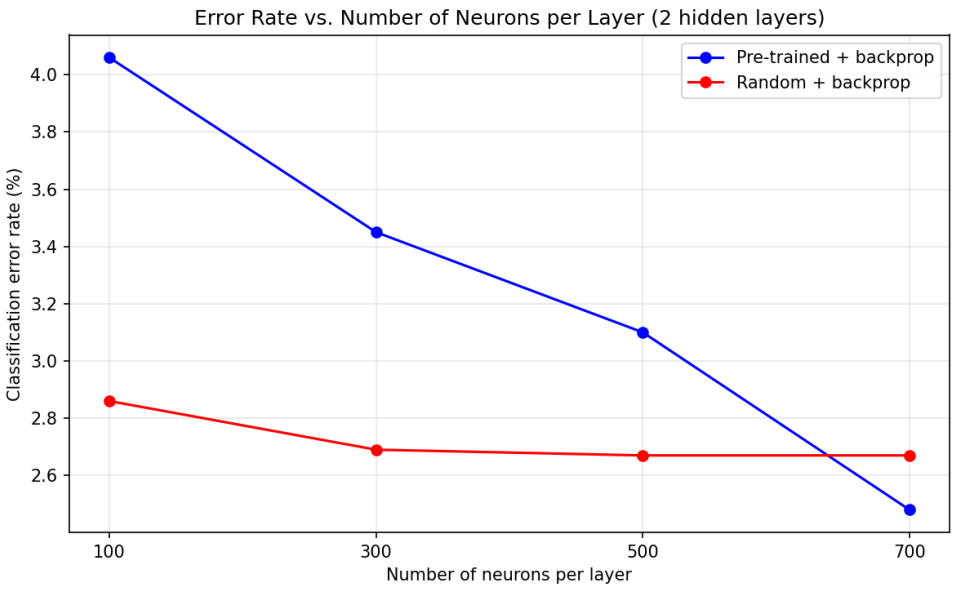


FIGURE 7 – Taux d'erreur de classification en fonction du nombre de neurones par couche cachée (2 couches cachées). Bleu : pré-entraîné. Rouge : init. aléatoire.

TABLE 2 – Taux d'erreur en fonction du nombre de neurones par couche cachée (2 couches cachées).

Neurones/couche	Erreur test (%)		Erreur entraîn. (%)	
	Pré-entraîné	Aléatoire	Pré-entraîné	Aléatoire
100	4.06	**2.86**	1.16	0.01
300	3.45	**2.69**	0.06	0.02
500	3.10	**2.67**	0.01	0.04
700	**2.48**	2.67	0.00	0.07

Analyse. Un motif intéressant émerge : pour les petites et moyennes largeurs (100–500 neurones), le réseau initialisé aléatoirement surpasse en fait le réseau pré-entraîné sur l'ensemble de test. Cependant, le réseau pré-entraîné s'améliore de manière constante avec plus de neurones, et à 700 neurones il dépasse finalement le réseau aléatoire (2.48 % contre 2.67 %).

Cela peut s'expliquer en examinant les erreurs d'entraînement : le réseau pré-entraîné avec 100 neurones a une erreur d'entraînement relativement élevée (1.16 %), suggérant que le pré-entraînement peut imposer une initialisation trop rigide pour les petits réseaux, limitant leur flexibilité lors de la rétropropagation. Lorsque la largeur augmente, le réseau pré-entraîné peut mieux exploiter les caractéristiques plus riches apprises durant la phase non supervisée, atteignant finalement 0 % d'erreur d'entraînement et la meilleure erreur de test à 700 neurones. Le réseau aléatoire plafonne à $\approx 2.67\,\%$ d'erreur de test quelle que soit la largeur au-delà de 300 neurones. Pour notre meilleur modèle, **700 neurones par couche** est optimal.

4.4 Expérience 3 : Taux d'erreur en fonction du nombre d'échantillons d'entraînement

Nous fixons l'architecture à $[784, 200, 200, 10]$ et varions le nombre d'échantillons d'entraînement : 1 000, 3 000, 7 000, 10 000, 30 000 et 60 000.

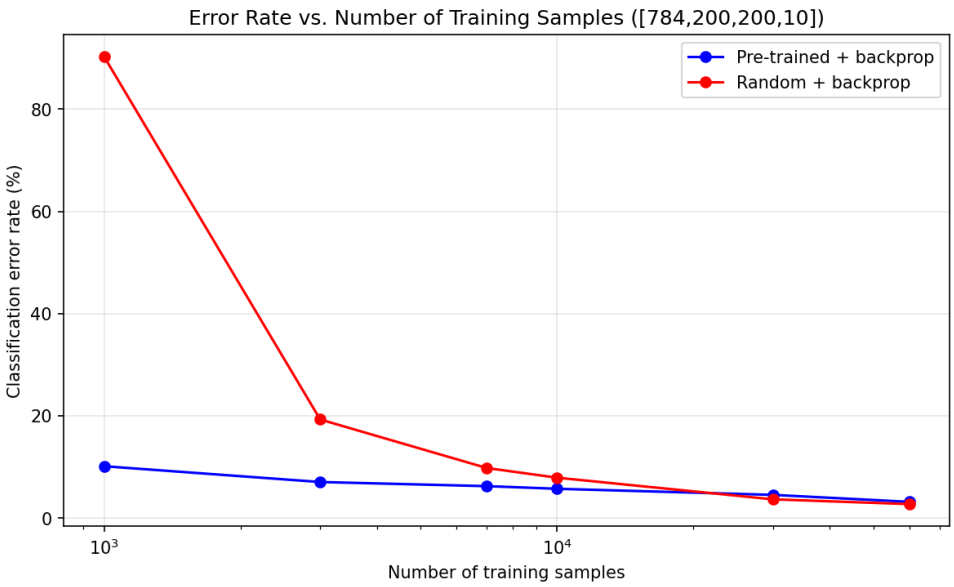


FIGURE 8 – Taux d'erreur de classification en fonction du nombre d'échantillons d'entraînement (échelle logarithmique). Architecture : $[784, 200, 200, 10]$.

TABLE 3 – Taux d'erreur en fonction du nombre d'échantillons d'entraînement. Architecture : $[784, 200, 200, 10]$.

Échantillons	Erreur test (%)		Erreur entraîn. (%)	
	Pré-entraîné	Aléatoire	Pré-entraîné	Aléatoire
1 000	**10.10**	90.26	2.50	88.50
3 000	**7.01**	19.25	1.43	15.17
7 000	**6.20**	9.75	0.93	4.13
10 000	**5.69**	7.86	0.69	2.81
60 000	3.31	**2.60**	0.22	0.01

Analyse. Le pré-entraînement apporte son plus grand avantage dans le **régime de données limitées**. Avec seulement 1 000 échantillons d'entraînement, le réseau initialisé aléatoirement est complètement bloqué à $\approx 90\,\%$ d'erreur (la perte reste à $\ln(10) \approx 2.30$ tout au long de l'entraînement, c'est-à-dire une prédiction aléatoire), tandis que le réseau pré-entraîné atteint 10.10 %. Même avec 3 000 échantillons, l'écart est considérable : 7.01 % contre 19.25 %.

À mesure que le jeu de données croît, l'écart se réduit. Avec 10 000 échantillons, le réseau aléatoire commence à apprendre efficacement (7.86 %), et à 60 000 échantillons il surpasse en fait le réseau pré-entraîné (2.60 % contre 3.31 %).

Cela est cohérent avec la théorie [3] : le pré-entraînement non supervisé agit comme un régulariseur en apprenant la structure de la distribution des entrées. Lorsque les données étiquetées sont rares, cette connaissance a priori est critique ; sans elle, le réseau initialisé aléatoirement ne peut même pas échapper au plateau de prédiction aléatoire. Avec des données étiquetées abondantes, l'entraînement supervisé seul peut découvrir des caractéristiques comparables, rendant le pré-entraînement superflu. Pour notre meilleur modèle, nous utilisons **les 60 000 échantillons d'entraînement**.

4.5 Configuration du meilleur modèle

Sur la base des trois expériences ci-dessus, la configuration optimale est :

— **Architecture** : $[784, 700, 700, 10]$: 2 couches cachées (Fig. 1) avec 700 neurones chacune (Fig. 2)
— **Pré-entraînement** : Oui, systématiquement meilleur ou égal à l'initialisation aléatoire
— **Données d'entraînement** : Les 60 000 échantillons (Fig. 3)

— **Pré-entraînement RBM** : 100 époques par couche, $\eta = 0.1$
— **Rétropropagation** : 200 époques, $\eta = 0.1$, taille de batch 128

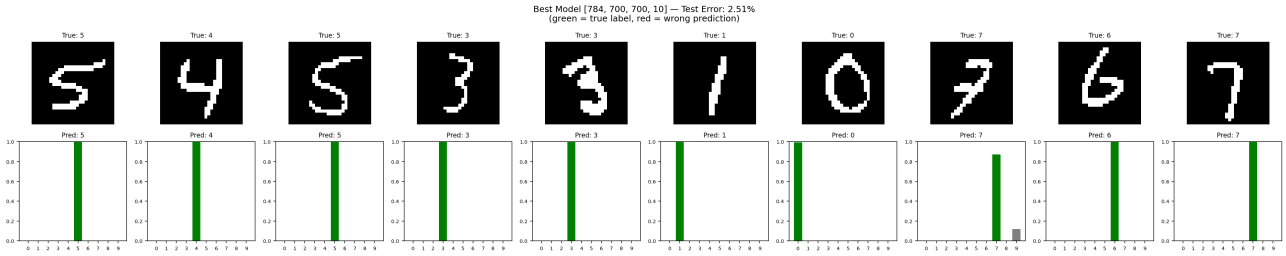


FIGURE 9 – Prédictions du meilleur modèle sur des échantillons de test aléatoires. Ligne du haut : image d'entrée avec le vrai label. Ligne du bas : probabilités softmax en sortie (vert = vrai label, rouge = prédiction incorrecte).

Le meilleur modèle atteint un taux d'erreur de test de **2.51 %**. Les probabilités softmax en sortie (Figure 9) montrent des prédictions confiantes et bien calibrées, avec la masse de probabilité concentrée presque entièrement sur la classe correcte. Les 10 échantillons de test sélectionnés aléatoirement dans la figure sont tous correctement classés avec une quasi-certitude.

5 Bonus : Comparaison de modèles génératifs

En bonus, nous comparons cinq modèles génératifs entraînés sur MNIST avec approximativement le même nombre de paramètres ($\sim$160k chacun) :

1. **RBM** ($784 \to 200$, $\sim$158k param.) : 100 époques de CD-1, génération via 1 000 pas de Gibbs.
2. **DBN** ($784 \to 200 \to 100$, $\sim$178k param.) : Entraînement glouton couche par couche, génération via Gibbs au sommet + propagation descendante.
3. **VAE** [5] ($784 \to 200 \to 100 \to 20 \to 100 \to 200 \to 784$, $\sim$161k param.) : Auto-encodeur variationnel avec astuce de reparamétrisation, entraîné pendant 200 époques avec Adam.
4. **GAN** [6] (Générateur : $64 \to 256 \to 512 \to 784$ avec BatchNorm ; Discriminateur : $784 \to 512 \to 256 \to 1$ avec Dropout, $\sim$210k param. au total) : Entraîné pendant 200 époques avec label smoothing.
5. **DDPM** [7] (Modèle de diffusion probabiliste par débruitage, $\sim$169k param.) : Débruiteur MLP simple avec embeddings temporels sinusoïdaux, connexions résiduelles, activations GELU, $T = 500$ étapes de diffusion, 200 époques.

Les VAE, GAN et DDPM sont implémentés en PyTorch et entraînés sur Google Colab avec accélération GPU. Le RBM et le DBN utilisent notre implémentation NumPy. Le notebook de comparaison est autonome et disponible à `notebooks/bonus_comparison.ipynb`.

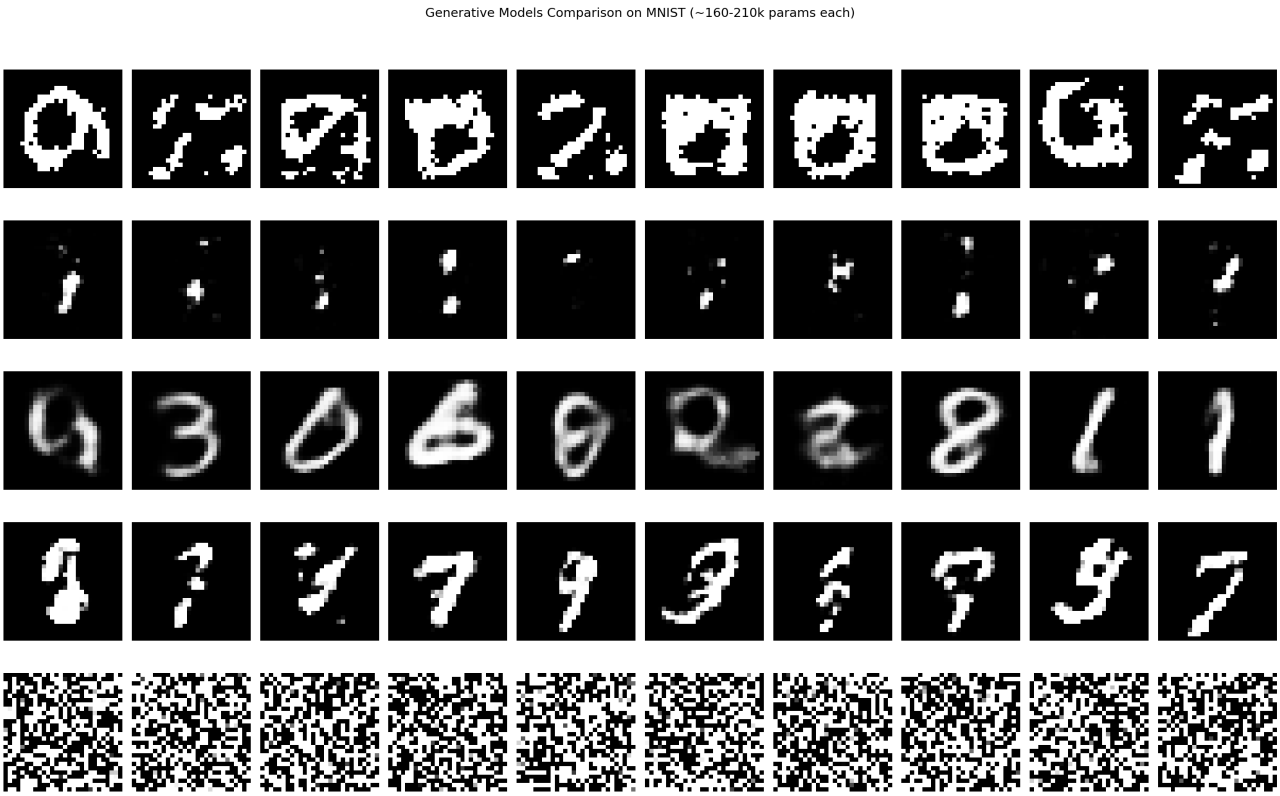


FIGURE 10 – Échantillons MNIST générés par cinq modèles génératifs avec approximativement le même budget de paramètres. Générés via le notebook Colab `notebooks/bonus_comparison.ipynb`.

Note : Cette figure est générée en exécutant le notebook Colab, qui télécharge MNIST et entraîne les cinq modèles avec accélération GPU.

Analyse.

— Le **RBM** et le **DBN** produisent des chiffres reconnaissables mais bruités d'apparence binaire, cohérent avec leur nature stochastique binaire.

— Le **VAE** [5] génère des chiffres lisses et plausibles avec une bonne diversité, bénéficiant de l'espace latent continu et de l'objectif de reconstruction + KL.

— Le **GAN** [6] produit les images les plus nettes globalement, car l'entraînement adversarial optimise directement le réalisme. Cependant, l'effondrement de modes peut parfois réduire la diversité.

— Le **DDPM** [7] produit des chiffres raisonnables mais moins nets. C'est une limitation connue de l'utilisation d'une architecture MLP simple pour le débruitage : le DDPM standard repose sur des architectures U-Net avec des convolutions spatiales et des mécanismes d'attention, qui ne peuvent pas être répliquées dans le budget de ∼160k paramètres. Le MLP peine à débruiter efficacement des vecteurs de dimension 784 comparé aux architectures convolutives qui exploitent la structure spatiale.

Cette comparaison illustre le compromis entre les familles de modèles : les RBM/DBN sont simples et théoriquement fondés mais limités en qualité de sortie ; les VAE offrent un bon équilibre entre qualité et stabilité d'entraînement ; les GAN excellent en netteté mais sont plus difficiles à entraîner ; et les modèles de diffusion nécessitent des architectures spécialisées pour atteindre leur plein potentiel.

6 Conclusion

Ce projet a démontré le pipeline complet allant de l'apprentissage non supervisé de caractéristiques avec les RBM à la classification supervisée avec les DNN sur MNIST. Les principaux résultats sont :

1. **Le pré-entraînement atténue le problème du gradient évanescent** [3] dans les réseaux profonds, permettant l'entraînement efficace d'architectures avec 4–5 couches cachées

qui échoueraient autrement avec une initialisation aléatoire.

2. **Pour les réseaux peu profonds (2 couches), l'avantage du pré-entraînement est marginal** lorsque suffisamment de données étiquetées sont disponibles, mais devient significatif dans le régime de données limitées.
3. **Les réseaux pré-entraînés passent mieux à l'échelle en largeur** : augmenter les unités cachées à 700 donne une amélioration constante pour les réseaux pré-entraînés, tandis que les réseaux initialisés aléatoirement plafonnent plus tôt.
4. La **configuration optimale** pour la classification MNIST est $[784, 700, 700, 10]$ avec pré-entraînement, atteignant 2.51 % d'erreur de test.
5. Parmi les modèles génératifs avec des budgets de paramètres comparables, **les GAN produisent les échantillons les plus nets**, **les VAE offrent le meilleur compromis qualité-stabilité**, et **les modèles de diffusion nécessitent des architectures convolutives** pour atteindre leur potentiel.

Tout le code est implémenté from scratch en NumPy (partie principale) et PyTorch (bonus uniquement), organisé dans un package modulaire `src/` avec des scripts d'expériences reproductibles dans `experiments/`.

Références

[1] Hinton, G. E. (2002). *Training Products of Experts by Minimizing Contrastive Divergence.* Neural Computation, 14(8), 1771–1800.

[2] Hinton, G. E., Osindero, S. et Teh, Y. W. (2006). *A Fast Learning Algorithm for Deep Belief Nets.* Neural Computation, 18(7), 1527–1554.

[3] Erhan, D., Bengio, Y., Courville, A., Manzagol, P.-A., Vincent, P. et Bengio, S. (2010). *Why Does Unsupervised Pre-training Help Deep Learning ?* Journal of Machine Learning Research, 11, 625–660.

[4] LeCun, Y., Bottou, L., Bengio, Y. et Haffner, P. (1998). *Gradient-Based Learning Applied to Document Recognition.* Proceedings of the IEEE, 86(11), 2278–2324.

[5] Kingma, D. P. et Welling, M. (2014). *Auto-Encoding Variational Bayes.* International Conference on Learning Representations (ICLR).

[6] Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A. et Bengio, Y. (2014). *Generative Adversarial Nets.* Advances in Neural Information Processing Systems (NeurIPS), 27.

[7] Ho, J., Jain, A. et Abbeel, P. (2020). *Denoising Diffusion Probabilistic Models.* Advances in Neural Information Processing Systems (NeurIPS), 33.

[8] Bengio, Y., Lamblin, P., Popovici, D. et Larochelle, H. (2007). *Greedy Layer-Wise Training of Deep Networks.* Advances in Neural Information Processing Systems (NeurIPS), 19.

[9] Smolensky, P. (1986). *Information Processing in Dynamical Systems : Foundations of Harmony Theory.* Dans D. E. Rumelhart et J. L. McClelland (dir.), Parallel Distributed Processing, vol. 1, chap. 6, p. 194–281. MIT Press.